

UNIVERSIDADE IGUAÇU - UNIG

Ciência Da Computação - 8º Período

Disciplina: Linguagens Formais e Compiladores

Aluna: Alessandra Brito Araújo

INCLUSALANG: LINGUAGEM FORMAL PARA SOLICITAÇÃO DE RECURSOS DE ACESSIBILIDADE E INCLUSÃO SOCIAL

Trabalho apresentado à disciplina de Linguagens Formais e Compiladores do curso de Ciência da Computação da Universidade Iguaçu – UNIG, como requisito para avaliação da atividade prática referente à especificação, implementação e validação de uma linguagem formal utilizando gramática BNF/EBNF, scanner léxico e parser descendente recursivo em Python.

1. Introdução

2. Problema

3. Objetivos

3.1 Objetivo Geral

3.2 Objetivos Específicos

4. Relação com os ODS

5. Especificação da Linguagem

5.1 Alfabeto

5.2 Tokens

5.3 Gramática BNF

5.4 Gramática EBNF

5.5 Autômato Finito Determinístico

6. Implementação

6.1 Scanner Léxico

6.2 Parser Descendente Recursivo

6.3 Interpretador

6.4 Interface Web

7. Casos de Teste

8. Resultados Obtidos

9. Conclusão

10. Referências

1. Introdução.

O projeto InclusaLang consiste no desenvolvimento de uma linguagem formal voltada ao registro de solicitações de recursos de acessibilidade e inclusão social. A proposta foi elaborada com o objetivo de aplicar os conceitos estudados na disciplina de Linguagens Formais e Compiladores, abrangendo a definição de uma gramática formal, análise léxica, análise sintática e interpretação de comandos.

A linguagem criada permite que usuários registrem solicitações de recursos de acessibilidade, realizem consultas e efetuem inscrições em atividades educacionais de forma estruturada e padronizada. Para validar o funcionamento da linguagem, foram desenvolvidos um scanner léxico, um parser utilizando a técnica de análise descendente recursiva e um interpretador, todos implementados na linguagem Python.

Além dos aspectos técnicos, o projeto está alinhado ao Objetivo de Desenvolvimento Sustentável (ODS) 10 da Organização das Nações Unidas (ONU), que busca reduzir as desigualdades e promover a inclusão social. Dessa forma, a solução proposta demonstra como os conceitos de Linguagens Formais podem ser aplicados na criação de ferramentas que apoiem iniciativas voltadas à acessibilidade e à igualdade de oportunidades.

2. Problema.

Pessoas em situação de vulnerabilidade frequentemente encontram dificuldades para acessar recursos de acessibilidade e oportunidades educacionais. Em muitas organizações e instituições, as solicitações desses recursos são realizadas de forma não padronizada, dificultando seu registro, acompanhamento e atendimento.

A ausência de um mecanismo estruturado para formalizar essas solicitações pode gerar inconsistências, retrabalho e atrasos na prestação do suporte necessário.

Diante desse cenário, torna-se importante desenvolver uma solução capaz de registrar e validar tais solicitações de maneira organizada e confiável.

Assim, o projeto InclusaLang foi concebido como uma linguagem formal específica para representar e processar solicitações relacionadas à acessibilidade e inclusão social, contribuindo para uma gestão mais eficiente dessas demandas.

3. Objetivos.

3.1 Objetivo Geral

Desenvolver uma linguagem formal denominada InclusaLang, voltada ao registro de solicitações de recursos de acessibilidade e inclusão social, aplicando conceitos de Linguagens Formais e Compiladores por meio da implementação de um scanner léxico, um parser descendente recursivo e um interpretador em Python.

3.2 Objetivos Específicos

Para alcançar o objetivo geral, foram definidos os seguintes objetivos específicos:

- Criar uma linguagem formal própria voltada ao contexto da inclusão social.
- Definir a gramática da linguagem utilizando as notações BNF e EBNF.
- Implementar um analisador léxico (scanner) para reconhecimento dos tokens.
- Implementar um analisador sintático (parser) utilizando a técnica de análise descendente recursiva.
- Desenvolver um interpretador capaz de executar os comandos válidos da linguagem.
- Realizar testes para validar o funcionamento da análise léxica, sintática e da interpretação dos comandos.
- Demonstrar a aplicação dos conceitos estudados na disciplina em um problema alinhado ao ODS 10.

4. Relação com os ODS.

ODS 10 – Redução das Desigualdades

O projeto InclusaLang está diretamente relacionado ao Objetivo de Desenvolvimento Sustentável 10 (ODS 10), que busca reduzir desigualdades e promover a inclusão social. A linguagem foi desenvolvida para registrar

solicitações de recursos de acessibilidade e participação em atividades educacionais, contribuindo para a criação de mecanismos que favoreçam a igualdade de oportunidades e o acesso a recursos de apoio.

Meta 10.2

Promover a inclusão social, econômica e política de todas as pessoas, independentemente de idade, deficiência, origem ou condição social.

A InclusaLang contribui para essa meta ao permitir que solicitações de recursos de acessibilidade sejam registradas de forma organizada e padronizada.

Meta 10.3

Garantir igualdade de oportunidades e reduzir desigualdades de resultados.

A proposta da linguagem busca facilitar o acesso a recursos que promovam a participação de pessoas em situação de vulnerabilidade em atividades educacionais e sociais.

ODS 4 – Educação de Qualidade

Além do ODS 10, o projeto também apresenta alinhamento com o ODS 4, que trata da promoção de uma educação inclusiva, equitativa e de qualidade.

Meta 4.5

Eliminar as disparidades de acesso à educação e garantir igualdade de oportunidades para todos.

Meta 4.a

Construir e adequar instalações educacionais que atendam às necessidades de todos os estudantes.

Por possibilitar o registro de solicitações relacionadas à acessibilidade e à participação em cursos e atividades educacionais, a InclusaLang atua como uma ferramenta de apoio à inclusão no ambiente educacional.

5. Especificação da Linguagem

A InclusaLang foi desenvolvida para representar solicitações de recursos de acessibilidade, consultas e inscrições em atividades educacionais por meio de uma sintaxe simples e padronizada.

5.1 Alfabeto

O alfabeto da linguagem InclusaLang é composto pelos seguintes elementos:

Palavras reservadas:

- INICIO
- FIM
- SOLICITAR
- CONSULTAR
- INSCREVER

Símbolos especiais:

- (
-)
- ;

Identificadores:

- ID

Os identificadores representam recursos de acessibilidade e atividades educacionais, tais como:

- RAMPA
- LEITOR_TELA
- CURSO_PROGRAMACAO

5.2 Tokens

Os tokens representam as unidades léxicas reconhecidas pelo scanner da linguagem. Durante a análise léxica, o código-fonte é dividido em elementos básicos que serão posteriormente utilizados pelo parser para validação sintática.

Token	Descrição
INICIO	Marca o início do programa
FIM	Marca o fim do programa
SOLICITAR	Solicita um recurso
CONSULTAR	Consulta um recurso
INSCREVER	Realiza inscrição em um curso
LPAREN	Símbolo "("
RPAREN	Símbolo ")"

Token	Descrição
PONTOVIRG	Símbolo ";"
ID	Identificador

Exemplo de Tokenização

Considere o seguinte comando da linguagem:

```
SOLICITAR(RAMPA);
```

Após a análise léxica, o scanner gera os seguintes tokens:

```
('SOLICITAR', 'SOLICITAR')
```

```
('LPAREN', '(')
```

```
('ID', 'RAMPA')
```

```
('RPAREN', ')')
```

```
('PONTOVIRG', ';')
```

Esses tokens são utilizados pelo parser para verificar se a estrutura do comando está de acordo com as regras gramaticais da linguagem.

5.3 Gramática BNF

A sintaxe da linguagem InclusaLang foi definida por meio da notação BNF (Backus-Naur Form), permitindo a descrição formal das estruturas aceitas pela linguagem.

```
<programa> ::= INICIO <lista_comandos> FIM
```

```
<lista_comandos> ::= <comando>
| <comando> <lista_comandos>
```

```
<comando> ::= SOLICITAR(<id>);
| CONSULTAR(<id>);
| INSCREVER(<id>);
```

<id> ::= ID

A gramática BNF apresentada define as regras sintáticas da linguagem InclusaLang. Todo programa deve iniciar com a palavra reservada INICIO e finalizar com FIM, podendo conter um ou mais comandos válidos entre esses delimitadores. Os comandos disponíveis são SOLICITAR, CONSULTAR e INSCREVER, os quais recebem um identificador como parâmetro.

5.4 Gramática EBNF

A EBNF (Extended Backus-Naur Form) permite representar a gramática de forma mais compacta e legível, utilizando recursos que simplificam a descrição das regras sintáticas da linguagem.

```
programa = "INICIO", lista_comandos, "FIM";
lista_comandos = { comando };
comando =
  "SOLICITAR", "(", id, ")", ";"
  | "CONSULTAR", "(", id, ")", ";"
  | "INSCREVER", "(", id, ")", ";";
id = ID;
```

Na representação EBNF, a expressão { comando } indica que a linguagem permite a repetição de comandos entre os delimitadores de início e fim do programa. Essa notação torna a gramática mais **compacta e legível quando comparada à representação em BNF.**

5.5 Autômato Finito Determinístico

O autômato finito determinístico (AFD) a seguir representa o fluxo básico de reconhecimento dos comandos válidos da linguagem InclusaLang.

(q0) -- INICIO --> (q1)

(q1) -- SOLICITAR --> (q2)

(q1) -- CONSULTAR --> (q2)

(q1) -- INSCREVER --> (q2)

(q2) -- (--> (q3)

(q3) -- ID --> (q4)

(q4) --) --> (q5)

(q5) -- ; --> (q1)

(q1) -- FIM --> (qf)

Descrição do Autômato

- **q0**: estado inicial do autômato.
- **q1**: reconhecimento do início do programa e espera por comandos.
- **q2**: reconhecimento de um comando da linguagem.
- **q3**: reconhecimento do parêntese de abertura.
- **q4**: reconhecimento de um identificador (ID).
- **q5**: reconhecimento do parêntese de fechamento e do ponto e vírgula.
- **qf**: estado final de aceitação.

O autômato inicia no estado **q0**, reconhece a palavra reservada **INICIO** e passa para o estado **q1**. Em seguida, os comandos da linguagem são processados pelos estados intermediários até que a palavra reservada **FIM** seja encontrada, levando o autômato ao estado final **qf**, indicando que o programa é válido de acordo com as regras definidas para a InclusaLang.

6. Implementação

6.1 Scanner Léxico

O scanner léxico foi desenvolvido em Python com o objetivo de identificar e classificar os elementos presentes nos programas escritos na linguagem InclusaLang. Durante essa etapa, o código-

fonte é analisado e convertido em uma sequência de tokens, que serão utilizados pelo analisador sintático.

Para o reconhecimento dos tokens foram utilizadas expressões regulares capazes de identificar palavras reservadas, símbolos especiais e identificadores da linguagem.

Exemplo

Entrada:

```
SOLICITAR(RAMPA);
```

Saída:

```
('SOLICITAR', 'SOLICITAR')  
( 'LPAREN', '(' )  
( 'ID', 'RAMPA' )  
( 'RPAREN', ')' )  
( 'PONTOVIRG', ';' )
```

Esses tokens são gerados pelo scanner léxico e representam os elementos básicos da linguagem. Em seguida, eles são enviados ao parser para validação sintática conforme as regras definidas pela gramática.

6.2 Parser Descendente Recursivo

O parser descendente recursivo é responsável por realizar a análise sintática dos programas escritos na linguagem InclusaLang. Após a geração dos tokens pelo scanner léxico, o parser verifica se a sequência de elementos reconhecidos segue corretamente as regras definidas pela gramática da linguagem.

A implementação foi realizada em Python utilizando a técnica de análise descendente recursiva, na qual cada regra gramatical é representada por uma função específica. Durante a execução, o parser percorre os tokens e valida a estrutura do programa, identificando possíveis erros sintáticos.

Quando a sequência de tokens segue corretamente as regras estabelecidas, a análise é concluída com sucesso. Caso

contrário, é apresentada uma mensagem indicando o erro encontrado e o elemento esperado.

Exemplo de Programa Válido

INICIO

SOLICITAR(RAMPA);

FIM

Resultado:

Programa sintaticamente válido!

Exemplo de Programa Inválido

INICIO

SOLICITAR(RAMPA)

FIM

Resultado:

Esperado PONTOVIRG

O parser garante que apenas programas que respeitem a gramática da InclusaLang sejam encaminhados para a etapa de interpretação.

6.3 Interpretador

O interpretador é o componente responsável pela execução dos comandos válidos reconhecidos pelo parser. Após a validação sintática do programa, os comandos são processados de acordo com suas respectivas funcionalidades.

A implementação do interpretador foi realizada em Python, permitindo registrar recursos solicitados e realizar consultas

sobre os recursos previamente armazenados durante a execução do programa.

O interpretador mantém internamente uma lista de recursos solicitados, possibilitando verificar sua existência quando comandos de consulta são executados.

Exemplo

Programa:

INICIO

SOLICITAR(RAMPA);

CONSULTAR(RAMPA);

CONSULTAR(LEITOR_TELA);

FIM

Resultado:

✅ Recurso solicitado: RAMPA

🔍 RAMPA foi solicitado.

❌ LEITOR_TELA não foi solicitado.

Dessa forma, o interpretador constitui a etapa final do processamento da linguagem, sendo responsável pela execução prática das instruções fornecidas pelo usuário.

6.4 Interface Web

Com o objetivo de facilitar a demonstração da linguagem InclusaLang, foi desenvolvida uma interface web utilizando Streamlit. A aplicação permite executar programas da linguagem, visualizar os tokens gerados pelo scanner léxico, validar a sintaxe por meio do parser descendente recursivo e acompanhar os resultados produzidos pelo interpretador. A interface também disponibiliza acesso à gramática BNF, à gramática EBNF, aos

Objetivos de Desenvolvimento Sustentável relacionados ao projeto e aos casos de teste definidos para validação da linguagem.

A aplicação também possui uma seção denominada "Aplicação Prática", que apresenta um cenário simulado de utilização da InclusaLang para registro e acompanhamento de solicitações de acessibilidade.

7. Casos de Teste

Para verificar o correto funcionamento da linguagem InclusaLang, foram realizados testes utilizando programas válidos e inválidos.

Caso de Teste 1 – Solicitação de Recurso

Entrada:

INICIO

SOLICITAR(RAMPA);

FIM

Resultado:

Programa sintaticamente válido!

 Recurso solicitado: RAMPA

Caso de Teste 2 – Consulta de Recurso Existente

Entrada:

INICIO

SOLICITAR(RAMPA);

CONSULTAR(RAMPA);

FIM

Resultado:

✅ Recurso solicitado: RAMPA

🔍 RAMPA foi solicitado.

Caso de Teste 3 – Consulta de Recurso Não Existente

Entrada:

INICIO

CONSULTAR(LEITOR_TELA);

FIM

``

Resultado:

❌ LEITOR_TELA não foi solicitado.

Caso de Teste 4 – Erro Sintático

Entrada:

INICIO

SOLICITAR(RAMPA)

FIM

Resultado:

Esperado PONTOVIRG

8. Resultados Obtidos

Os resultados obtidos demonstraram que a linguagem InclusaLang foi capaz de executar corretamente as etapas de análise léxica, análise sintática e interpretação dos comandos definidos em sua gramática.

O scanner identificou adequadamente os tokens da linguagem, o parser validou a estrutura sintática dos programas e o interpretador executou as instruções válidas, permitindo o registro e a consulta de recursos de acessibilidade.

Os testes realizados comprovaram o funcionamento correto da implementação e a capacidade do sistema de detectar erros sintáticos durante o processamento dos comandos.

9. Conclusão

O desenvolvimento da linguagem InclusaLang possibilitou a aplicação prática dos conceitos estudados na disciplina de Linguagens Formais e Compiladores. Durante o projeto foram definidas regras gramaticais em BNF e EBNF, implementado um scanner léxico, desenvolvido um parser descendente recursivo e construído um interpretador funcional em Python.

Além dos aspectos técnicos, o projeto foi alinhado aos Objetivos de Desenvolvimento Sustentável da Organização das Nações Unidas, especialmente ao ODS 10, relacionado à redução das desigualdades, e ao ODS 4, voltado à promoção de uma educação inclusiva e acessível.

Os objetivos propostos foram alcançados com sucesso, resultando em uma linguagem formal funcional, capaz de representar solicitações de acessibilidade de forma organizada, estruturada e validada por mecanismos de análise léxica e sintática.

10. Referências

BRASIL. Ministério da Educação. Base Nacional Comum Curricular (BNCC). Brasília: MEC, 2018.

HOPCROFT, John E.; MOTWANI, Rajeev; ULLMAN, Jeffrey D. *Introdução à Teoria de Autômatos, Linguagens e Computação*. 3. ed. Rio de Janeiro: Elsevier, 2002.

LOUDEN, Kenneth C. *Compiladores: Princípios e Práticas*. São Paulo: Cengage Learning, 2004.

ORGANIZAÇÃO DAS NAÇÕES UNIDAS (ONU). Objetivos de Desenvolvimento Sustentável. Disponível em: <https://brasil.un.org>. Acesso em: 03 set. 2026.

PYTHON SOFTWARE FOUNDATION. Python Documentation. Disponível em: <https://docs.python.org>. Acesso em: 03 set. 2026.